



UNIVERSIDAD AUTÓNOMA DE CHIHUAHUA

Facultad de Ingeniería

BASES DE DATOS
AVANZADAS

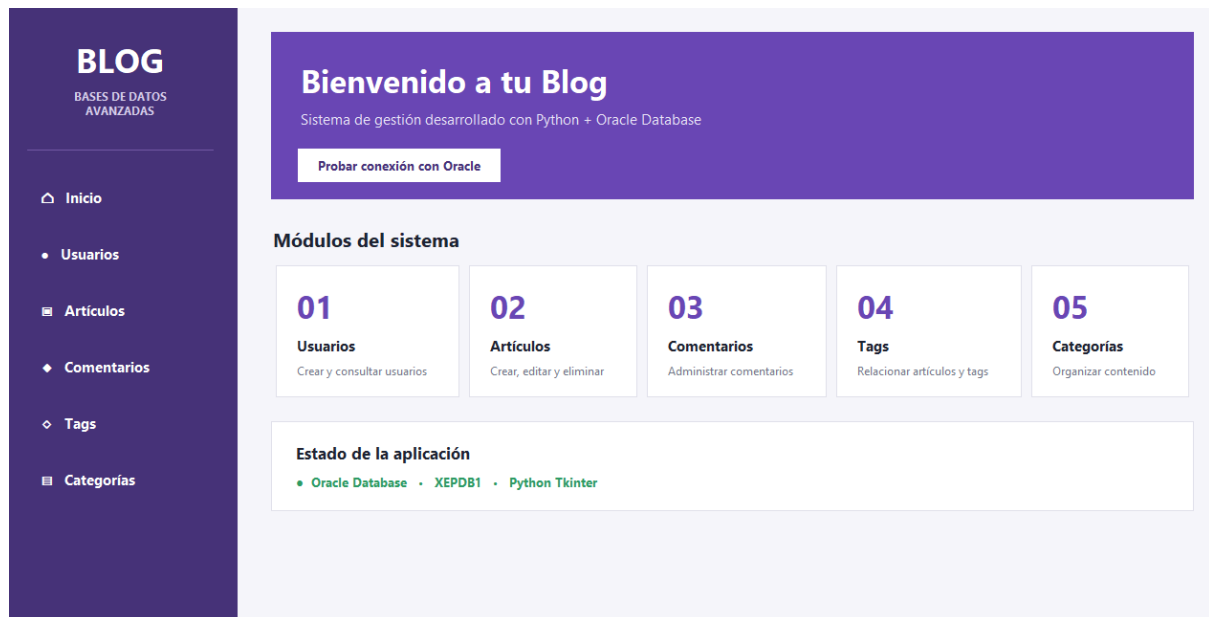
PROYECTO PRIMER PARCIAL BLOG

Jose Saul Delira Miramontes

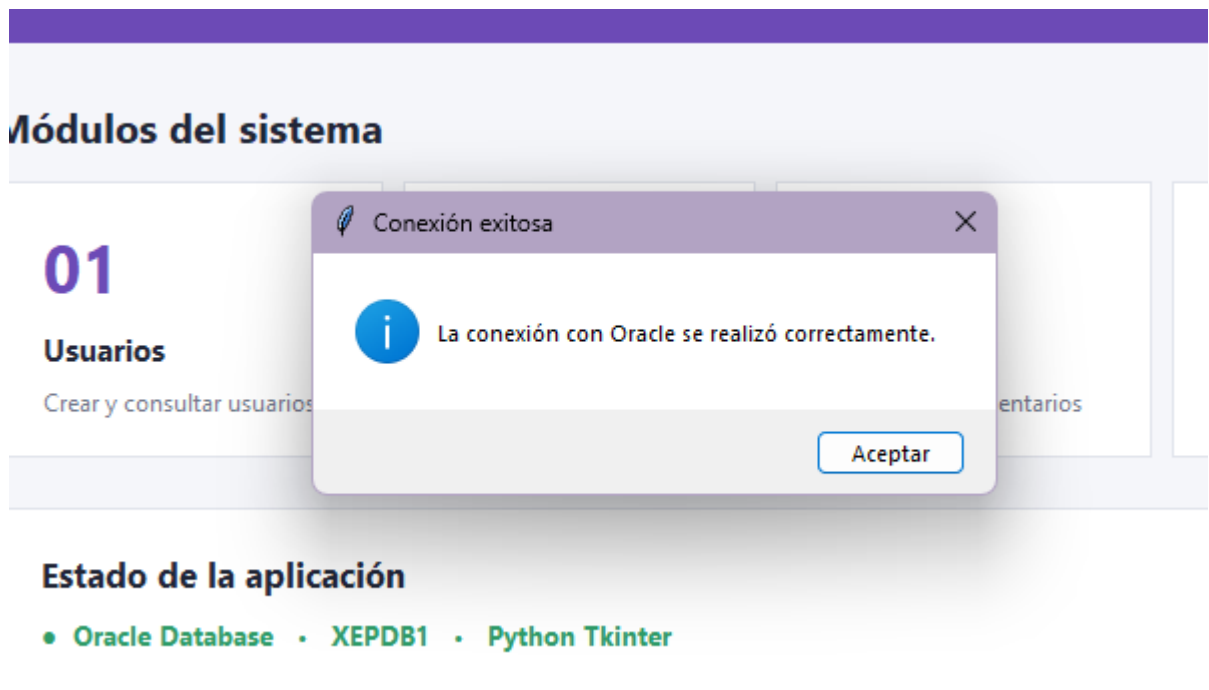
Ana Ruvi Morales Chavez 376899
Mauricio Adrian Ochoa Carrillo 377194

7CC2

Pantalla inicial tenemos del lado izquierdo el menu el cual controla todo



se prueba la conexión a la base de datos antes de iniciar cualquier procedimiento para evitar posibles errores



una vez que se comprobó la base de datos podemos continuar con nuestra aplicación la primera partes es la de usuarios

BLOG
BASES DE DATOS
AVANZADAS

Inicio

Usuarios

Articulos

Comentarios

Tags

Categorias

Usuarios

Administra los usuarios registrados en el sistema.

Crear usuario

Nombre:

Email:

Crear usuario

Consultar usuario

ID usuario:

Consultar

aqui tenemos las opciones de crear o consultar usuarios para este caso vamos a registrar a un usuario y luego consultarlo tanto aqui en la aplicación como en la base de datos para confirmar que esta funcionando correctamente

Usuarios

Administra los usuarios registrados en el sistema.

Crear usuario

Nombre:

Mauricio Ochoa

Email:

a377194@uach.mx

Crear usuario

Consultar usuario

ID usuario:

Usuario creado

Usuario creado correctamente.
ID: 21

Aceptar

Usuarios

Administra los usuarios registrados en el sistema.

Crear usuario

Nombre:

Email:

Crear usuario

Consultar usuario

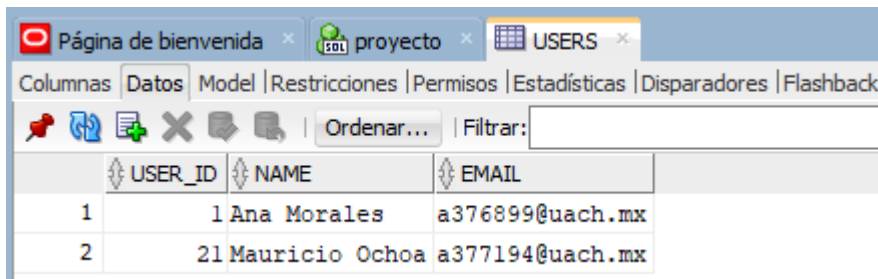
ID usuario:

21

Consultar

Nombre: Mauricio Ochoa | Email: a377194@uach.mx

vamos a la base de datos a comprobar si se guardo correctamente el nuevo usuario



	USER_ID	NAME	EMAIL
1	1	Ana Morales	a376899@uach.mx
2	21	Mauricio Ochoa	a377194@uach.mx

Despues tenemos la parte de articulos

Artículos

Crea, consulta, actualiza, elimina y ordena los artículos.

Datos del artículo

ID artículo:

1

ID usuario:

21

Título:

Rutina para hiperpigmentación

Texto:

Combina Thiamidol, niacinamida y ácidos. No se usan todos el mismo día para no irritar

Crear

Consultar

Actualizar

Eliminar

Limpiar

Ordenar por:

Título

Tipo:

Descendente

Ordenar

Mostrar todos

ID	Título	Fecha	Usuario

para este caso se insertaron 8 articulos pero un caso se comprobo que funcionara todo e intentamos registrar un producto sin id de usuario y no dejo

BLOG

BASES DE DATOS AVANZADAS

- Inicio
- Usuarios
- Artículos
- Comentarios
- Tags
- Categorías

Python • Oracle

Proyecto académico

Artículos

Crea, consulta, actualiza, elimina y ordena los artículos.

Datos del artículo

ID artículo: ID usuario:

Título:

Texto:

Ordenar por: Tipo:

ID	Título	Fecha	Usuario
7	Qué es Thiamidol	24/09/2026 20:44	1
6	Salicílico vs Glicólico	24/09/2026 20:42	21
5	Cómo recuperar la barrera	24/09/2026 20:41	1

Error

invalid literal for int() with base 10: "

Aceptar

vamos a la base de datos a ver la tabla de artículos

ARTICLE_ID	USER_ID	TITLE	ARTICLE_DATE	TEXT
1	1	21 Rutina para hiperpigmentación	24/09/26	Combina Thiamidol, niacinamida y ácidos. No se usan todos el mismo día para no irritar
2	2	21 Niacinamida vs Glicólico	24/09/26	Niacinamida controla sebo y barrera. Glicólico exfolia. Juntos pueden irritar si no se o
3	3	1 Por qué el SPF 50+ es obligatorio	24/09/26	En clima con sol fuerte, sin SPF pierdes en una semana lo que logras en un mes de tratam
4	4	1 Señales de barrera dañada	24/09/26	Ardor, piel brillante y sensibilidad a todo. Es por exceso de ácidos. Hay que parar, no
5	5	1 Cómo recuperar la barrera	24/09/26	Pausa ácidos 1-2 semanas. Solo limpiador suave, humectante y SPF. Nada nuevo hasta mejor
6	6	21 Salicílico vs Glicólico	24/09/26	Salicílico entra al poro y es para grasa y puntos negros. Glicólico es para textura y to
7	7	1 Qué es Thiamidol	24/09/26	Inhibe la melanina con menos irritación que la hidroquinona. Por eso es más caro.
8	8	21 Rutina de día vs noche	24/09/26	Día: proteger con antioxidante y SPF. Noche: reparar con ácidos o retinoides.

hacemos una prueba de eliminacion con un articulo que no existe y despues con uno que si existe para comprobar el funcionamiento total

Datos del artículo

ID artículo: ID usuario:

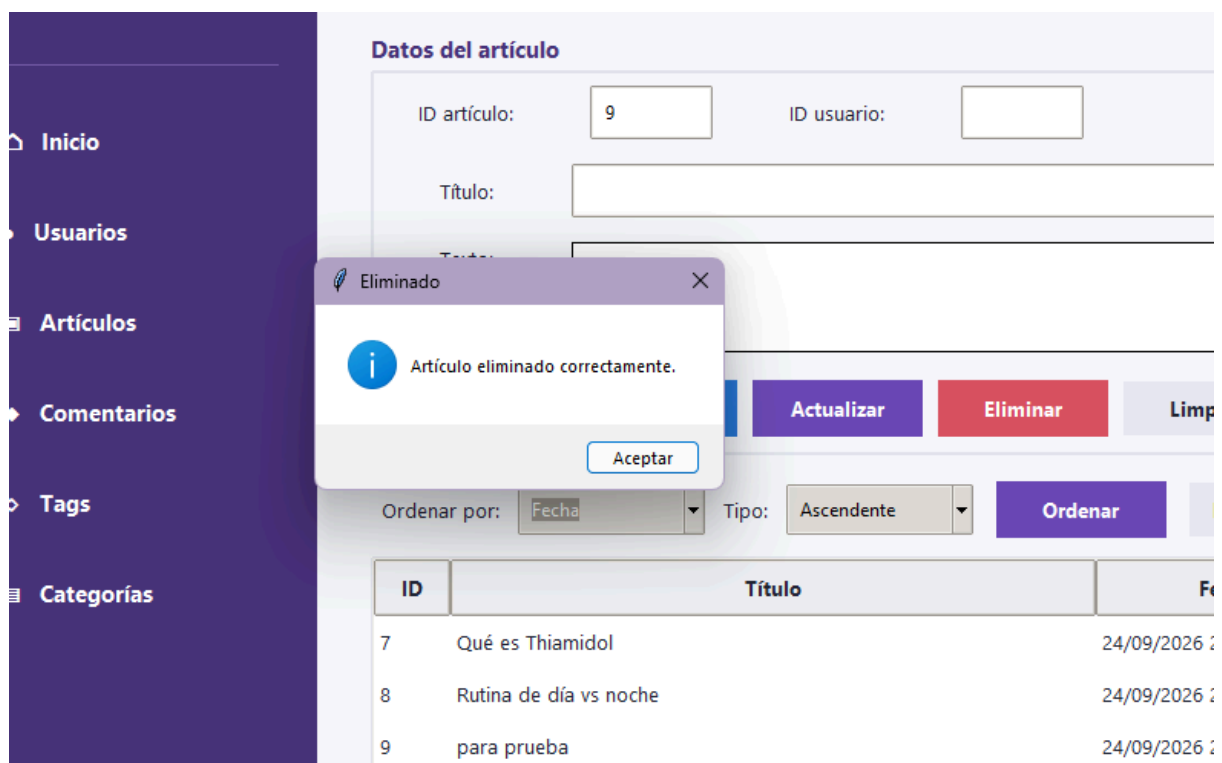
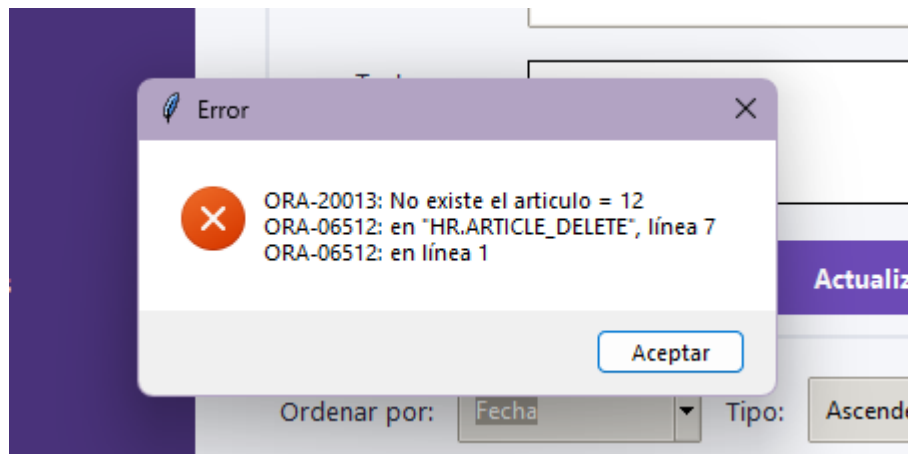
Título:

Texto:

Ordenar por: Tipo:

Confirmar

¿Desea eliminar el artículo 12?



continuamos con la parte de comentarios para estos casos no pondremos un URL

Comentarios

Agrega y consulta los comentarios asociados a un artículo.

Agregar comentario

ID artículo: Nombre: URL:

Agregar comentario

Consulta por ID de artículo

Ver comentarios

Comentario

Comentario agregado.
ID: 1

Aceptar

ID	Nombre	URL	Fecha
----	--------	-----	-------

proyecto		COMMENTS			
Columnas Datos Model Restricciones Permisos Estadísticas Disparadores Flashback Dependencias Detalles Particiones Índices SQL					
Ordenar... Filtrar:					
	COMMENT_ID	ARTICLE_ID	NAME	URL	COMMENT_DATE
1	1	1	Gael R	(null)	24/09/26
2	2	2	1 Gael R	(null)	24/09/26
3	3	3	1 Jacqueline H	(null)	24/09/26

luego pasamos a la parte de tags

BLOG

BASES DE DATOS AVANZADAS

- Inicio
- Usuarios
- Artículos
- Comentarios
- Tags
- Categorías

Tags

Crea tags, relacionalos con artículos y realiza búsquedas.

Crear tag

Nombre: URL:

Crear tag

Asociar tag a artículo

ID artículo:

Asociar

Consultar tags de un artículo

ID artículo:

Ver tags

ID	Nombre	URL
----	--------	-----

Tag creado

Tag creado correctamente.
ID: 1

Aceptar

Tags

Crea tags, relacionalos con artículos y realiza búsquedas.

Crear tag

Nombre: URL:

Crear tag

Asociar tag a artículo

ID artículo: ID tag:

Asociar

Consultar tags de un artículo

ID artículo:

Ver tags

ID	Nombre	URL
----	--------	-----

Asociación

Tag asociado correctamente.

Aceptar

proyecto.sql	Página de bienvenida	proyecto	ARTICLE_TAGS
Columnas	Datos	Model	Restricciones
Permisos	Estadísticas	Disparadores	Flashback
Dependencias	Ordenar...	Filtrar:	
ARTICLE_ID	TAG_ID		
1	1	1	
2	1	2	
3	2	3	
4	3	5	
5	2	4	
6	8	2	

en la parte de categorías ya tenía registradas las categorías por lo cual solo muestro la parte de asociar

Categorías

Organiza los artículos mediante categorías.

Crear categoría

Nombre:

Asociar categoría

ID artículo:
ID categoría:

Asociación

i

Categoría asociada correctamente.

ARTICLE_ID	CATEGORY_ID
1	1
2	3

al final de todos esas modificaciones la base de datos se quedo asi
tabla usuarios:

USER_ID	NAME	EMAIL
1	1 Ana Morales	a376899@uach.mx
2	21 Mauricio Ochoa	a377194@uach.mx

tabla articulos:

```
--para articulos
SELECT * FROM articles;
```

Resultado de la Consulta x

Todas las Filas Recuperadas: 8 en 0.028 segundos

ARTICLE_ID	USER_ID	TITLE	ARTICLE_DATE	TEXT
1	1	21 Rutina para hiperpigmentación	24/09/26	Combina Thiamidol, niacinamida y ácidos. No se usan todos el mismo día para no i
2	2	21 Niacinamida vs Glicólico	24/09/26	Niacinamida controla sebo y barrera. Glicólico exfolia. Juntos pueden irritar si
3	3	1 Por qué el SPF 50+ es obligatorio	24/09/26	En clima con sol fuerte, sin SPF pierdes en una semana lo que logras en un mes d
4	4	1 Señales de barrera dañada	24/09/26	Ardor, piel brillante y sensibilidad a todo. Es por exceso de ácidos. Hay que pa
5	5	1 Cómo recuperar la barrera	24/09/26	Pausa ácidos 1-2 semanas. Solo limpiador suave, humectante y SPF. Nada nuevo has
6	6	21 Salicilico vs Glicólico	24/09/26	Salicilico entra al poro y es para grasa y puntos negros. Glicólico es para text
7	7	1 Qué es Thiamidol	24/09/26	Inhibe la melanina con menos irritación que la hidroquinona. Por eso es más caro
8	8	21 Rutina de día vs noche	24/09/26	Día: proteger con antioxidante y SPF. Noche: reparar con ácidos o retinoides.

tabla comentarios:

```
--para cometarios
SELECT * FROM comments;
```

Resultado de la Consulta x

Todas las Filas Recuperadas: 3 en 0.033 segundos

COMMENT_ID	ARTICLE_ID	NAME	URL	COMMENT_DATE
1	1	1 Gael R	(null)	24/09/26
2	2	1 Gael R	(null)	24/09/26
3	3	1 Jacqueline H	(null)	24/09/26

tabla tags:

```
--para tags
SELECT * FROM tags;
```

Resultado de la Consulta x

Todas las Filas Recuperadas: 6 en 0.048 segundos

TAG_ID	NAME	URL
1	1 hiperpigmentacion	(null)
2	2 rutina	(null)
3	3 acidos	(null)
4	4 ingredientes	(null)
5	5 spf	(null)
6	6 barrera	(null)

tabla categorias:

```
--para categoria  
SELECT * FROM categories;
```

Resultado de la Consulta x

Todas las Filas Recuperadas: 1 en 0.012 segundos

	CATEGORY_ID	NAME	URL
1	1	Rutona	www.desconocido.com

tabla articulos-tags:

```
--para la relacion de articulos - tags  
SELECT * FROM article_tags;
```

Resultado de la Consulta x

Todas las Filas Recuperadas: 6 en 0.012 segundos

	ARTICLE_ID	TAG_ID
1	1	1
2	1	2
3	2	3
4	2	4
5	3	5
6	8	2

tabla articulos - categoria:

```
--para la realcion articulos - categorias  
SELECT * FROM article_categories;
```

Resultado de la Consulta x

Todas las Filas Recuperadas: 2 en 0.011 segundos

	ARTICLE_ID	CATEGORY_ID
1	1	1
2	3	1

Documentacion:

TABLAS

```
CREATE TABLE users (  
    user_id NUMBER,  
    name VARCHAR2(100) NOT NULL,  
    email VARCHAR2(150) NOT NULL,  
    CONSTRAINT pk_users PRIMARY KEY (user_id)  
);  
CREATE UNIQUE INDEX uq_users_email ON users (LOWER(email));
```

Se crea la tabla users para almacenar los usuarios registrados en el blog. Contiene tres columnas: user_id de tipo NUMBER, que representa el identificador único de cada usuario. name de tipo VARCHAR2(100), que guarda el nombre del usuario y es obligatorio (NOT NULL), y email de tipo VARCHAR2(150), que guarda el correo electrónico y también es obligatorio (NOT NULL). En la columna user_id se pone el constraint pk_users de tipo PRIMARY KEY, lo que hace que no existan dos usuarios con el mismo ide y que este nunca quede vacío. Además, se crea un índice llamado uq_users_email en la función LOWER(email). Esto sirve para que dos usuarios no puedan tener el mismo correo electrónico, sin importar si escriben mayúsculas o minúsculas, por ejemplo, Juan@X.com y juan@x.com con esto se consideran iguales los 2

```
CREATE TABLE articles (  
    article_id NUMBER,  
    user_id NUMBER NOT NULL,  
    title VARCHAR2(200) NOT NULL,  
    article_date DATE NOT NULL,  
    article_text VARCHAR2(4000),  
    CONSTRAINT pk_articles PRIMARY KEY (article_id),  
    CONSTRAINT fk_articles_user FOREIGN KEY (user_id) REFERENCES users (user_id)  
);
```

Se crea la tabla articles para almacenar los artículos publicados en el blog. Tiene cinco columnas: article_id de tipo NUMBER, el identificador único de cada artículo, user_id de tipo NUMBER, el usuario autor del artículo y es obligatorio (NOT NULL), title de tipo VARCHAR2(200), que guarda el título del artículo y es obligatorio. article_date de tipo DATE, que guarda la fecha del artículo y es obligatoria; y article_text de tipo VARCHAR2(4000), que guarda el contenido del artículo y es opcional. En la columna article_id se define el constraint pk_articles de tipo PRIMARY KEY, que hace que no existan dos artículos con el mismo identificador y que este no quede vacío. Además, en la columna user_id se define el constraint fk_articles_user de tipo FOREIGN KEY, que hace referencia a la columna user_id de

la tabla users. Esto hace que todo artículo tenga un autor registrado y evita que se cree un artículo con un usuario inexistente.

```
CREATE TABLE comments (  
    comment_id NUMBER,  
    article_id NUMBER NOT NULL,  
    name VARCHAR2(100) NOT NULL,  
    url VARCHAR2(300),  
    comment_date DATE NOT NULL,  
    CONSTRAINT pk_comments PRIMARY KEY (comment_id),  
    CONSTRAINT fk_comments_article FOREIGN KEY (article_id) REFERENCES articles  
(article_id)  
);
```

Creemos la tabla comments para almacenar los comentarios que los visitantes dejan en los artículos. Contiene cinco columnas: comment_id de tipo NUMBER, que representa el identificador único del comentario; article_id de tipo NUMBER, que indica el artículo al que pertenece el comentario y es obligatorio (NOT NULL); name de tipo VARCHAR2(100), que guarda el nombre de quien comenta y es obligatorio; url de tipo VARCHAR2(300), que guarda un enlace opcional asociado al comentario; y comment_date de tipo DATE, que guarda la fecha del comentario y es obligatoria. Sobre la columna comment_id se define el constraint pk_comments de tipo PRIMARY KEY, siguiendo el mismo patrón que en las tablas anteriores. Sobre la columna article_id se define el constraint fk_comments_article de tipo FOREIGN KEY, que hace referencia a article_id de la tabla articles, asegurando que todo comentario pertenezca a un artículo existente.

```
CREATE TABLE tags (  
    tag_id NUMBER,  
    name VARCHAR2(100) NOT NULL,  
    url VARCHAR2(300),  
    CONSTRAINT pk_tags PRIMARY KEY (tag_id)  
);  
CREATE UNIQUE INDEX uq_tags_name ON tags (LOWER(name));
```

Se crea la tabla tags para almacenar las etiquetas que se pueden asociar a los artículos. Contiene tres columnas: tag_id de tipo NUMBER, que representa el identificador único de la etiqueta. name de tipo VARCHAR2(100), que guarda el nombre de la etiqueta y es obligatorio (NOT NULL). y url de tipo VARCHAR2(300), que guarda un enlace asociado a la etiqueta. En la columna tag_id se define el constraint pk_tags de tipo PRIMARY KEY. Además, se crea un índice único llamado uq_tags_name sobre LOWER(name), con la misma idea que en la tabla users

```
CREATE TABLE article_tags (
  article_id NUMBER NOT NULL,
  tag_id    NUMBER NOT NULL,
  CONSTRAINT pk_article_tags PRIMARY KEY (article_id, tag_id),
  CONSTRAINT fk_at_article FOREIGN KEY (article_id) REFERENCES articles
(article_id),
  CONSTRAINT fk_at_tag    FOREIGN KEY (tag_id)    REFERENCES tags (tag_id)
);
```

En esta tabla hay 2 columnas: article_id de tipo NUMBER y tag_id de tipo NUMBER, ambas obligatorias (NOT NULL). En esta tabla tenemos 2 identificadores, se define el constraint pk_article_tags de tipo PRIMARY KEY hecha por las columnas (article_id, tag_id), lo que hace que un mismo artículo no pueda tener la misma etiqueta repetida. En article_id se define el constraint fk_at_article de tipo FOREIGN KEY hacia articles (article_id), y sobre tag_id se define el constraint fk_at_tag de tipo FOREIGN KEY hacia tags (tag_id). Para que solo se puedan asociar artículos y etiquetas que existan previamente.

```
CREATE TABLE categories (
  category_id NUMBER,
  name        VARCHAR2(100) NOT NULL,
  url         VARCHAR2(300),
  CONSTRAINT pk_categories PRIMARY KEY (category_id)
);
CREATE UNIQUE INDEX uq_categories_name ON categories (LOWER(name));
```

Se crea la tabla categories para almacenar las categorías que se pueden asociar a los artículos. Tiene tres columnas: category_id de tipo NUMBER, que representa el identificador único de la categoría. name de tipo VARCHAR2(100), que guarda el nombre de la categoría y es obligatorio (NOT NULL). y url de tipo VARCHAR2(300), que guarda un enlace. En la columna category_id se define el constraint pk_categories de tipo PRIMARY KEY. Además, se crea un índice único llamado uq_categories_name sobre LOWER(name), con la misma idea que en las tablas users y tags.

```
CREATE TABLE article_categories (
  article_id  NUMBER NOT NULL,
  category_id NUMBER NOT NULL,
  CONSTRAINT pk_article_categories PRIMARY KEY (article_id, category_id),
  CONSTRAINT fk_ac_article FOREIGN KEY (article_id) REFERENCES articles
(article_id),
  CONSTRAINT fk_ac_category FOREIGN KEY (category_id) REFERENCES categories
(category_id)
```

);

En esta tabla hay 2 columnas: `article_id` de tipo `NUMBER` y `category_id` de tipo `NUMBER`, ambas obligatorias (`NOT NULL`). En esta tabla tenemos 2 identificadores, se define el constraint `pk_article_categories` de tipo `PRIMARY KEY` hecha por las columnas (`article_id`, `category_id`), lo que hace que un mismo artículo no pueda tener la misma categoría repetida. En `article_id` se define el constraint `fk_ac_article` de tipo `FOREIGN KEY` hacia `articles (article_id)`, y sobre `category_id` se define el constraint `fk_ac_category` de tipo `FOREIGN KEY` hacia `categories (category_id)`.

USUARIOS

```
CREATE OR REPLACE PROCEDURE user_insert (  
    p_name IN users.name%TYPE,  
    p_email IN users.email%TYPE,  
    p_id OUT users.user_id%TYPE  
)  
IS  
BEGIN  
    SELECT NVL(MAX(user_id), 0) + 1 INTO p_id FROM users;  
  
    INSERT INTO users (user_id, name, email)  
    VALUES (p_id, p_name, p_email);  
END;  
/
```

Recibe tres parámetros: `p_name` de tipo `users.name%TYPE`, que es de entrada (IN) y corresponde al nombre del usuario. `p_email` de tipo `users.email%TYPE`, también de entrada. y `p_id` de tipo `users.user_id%TYPE`, que es de salida (OUT) y devolverá el identificador generado.

Dentro del bloque, primero se calcula el nuevo identificador con la sentencia `SELECT NVL(MAX(user_id), 0) + 1 INTO p_id FROM users`. La función `MAX` obtiene el valor más alto actual de `user_id`, si la tabla está vacía, `MAX` devuelve `NULL`, por eso se usa `NVL` para convertir ese `NULL` en 0, y luego se suma 1. Con esto el primer usuario tendrá `user_id` 1 y los siguientes irán incrementando. El resultado se guarda en el parámetro de salida `p_id` mediante la cláusula `INTO`.

Después se ejecuta la sentencia `INSERT INTO users (user_id, name, email) VALUES (p_id, p_name, p_email)`, que inserta una nueva fila en la tabla `users` con el identificador calculado, el nombre y el correo recibidos como parámetros.

```

CREATE OR REPLACE PROCEDURE user_get (
    p_id    IN users.user_id%TYPE,
    p_name  OUT users.name%TYPE,
    p_email OUT users.email%TYPE
)
IS
BEGIN
    SELECT name, email
    INTO p_name, p_email
    FROM users
    WHERE user_id = p_id;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        raise_application_error(-20001, 'El usuario ' || p_id || ' no existe.');
```

END;

/

El procedimiento user_get lo usamos para obtener el nombre y el correo de un usuario a partir de su id. Recibe tres parámetros: p_id de tipo users.user_id%TYPE, es de entrada y se usa id porque es lo que vamos a buscar, p_name de tipo users.name%TYPE, que es de salida y devolverá el nombre del usuario, y p_email de tipo users.email%TYPE, que también es de salida y devolverá el correo electrónico.

En el bloque se ejecuta la sentencia SELECT name, email INTO p_name, p_email FROM users WHERE user_id = p_id. Lo que hace aquí es buscar los usuarios que coincidan con los parametros que entregamos, y guarda los valores de name y email en p_name y p_email.

Si el usuario no existe, el SELECT INTO no devuelve ninguna fila y Oracle lanza automáticamente la excepción NO_DATA_FOUND. Se usa la EXCEPTION con WHEN NO_DATA_FOUND THEN, luego usamos a raise_application_error con el código -20001 y luego lanza un mensaje

```

CREATE OR REPLACE FUNCTION fn_existe_email (
    p_email IN users.email%TYPE
) RETURN NUMBER
IS
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM users
    WHERE LOWER(email) = LOWER(p_email);
    RETURN v_count;
END;
```

/

Se crea la función `fn_existe_email` para ver si un correo electrónico ya está registrado en la tabla `users`. Recibe un parámetro `p_email` de tipo `users.email%TYPE`, que es de entrada y es correo que se quiere comprobar. Antes del `BEGIN` se declara la variable local `v_count` de tipo `NUMBER`, que se usará para guardar el resultado del conteo.

La función `COUNT` cuenta cuántas filas de la tabla `users` cumplen la condición. La comparación se hace con `LOWER` en ambos lados para que no importen las mayúsculas o minúsculas. El resultado del conteo se guarda en la variable `v_count`. Como `COUNT` siempre devuelve una fila, aquí no se necesita manejar la excepción `NO_DATA_FOUND`.

Finalmente se ejecuta `RETURN v_count`, que devuelve el valor al llamador. Si el correo ya existe, devuelve 1. Si no existe, devuelve 0.

ARTÍCULOS

```
CREATE OR REPLACE PROCEDURE article_insert (  
    p_user_id IN articles.user_id%TYPE,  
    p_title   IN articles.title%TYPE,  
    p_text    IN articles.article_text%TYPE,  
    p_id      OUT articles.article_id%TYPE  
)  
IS  
BEGIN  
    SELECT NVL(MAX(article_id), 0) + 1 INTO p_id FROM articles;  
  
    INSERT INTO articles (article_id, user_id, title, article_date, article_text)  
    VALUES (p_id, p_user_id, p_title, sysdate, p_text);  
END;  
/
```

Se crea el procedimiento `article_insert` para insertar un nuevo artículo en la tabla `articles`. Recibe cuatro parámetros: `p_user_id` de tipo `articles.user_id%TYPE`, que es de entrada y es el id del usuario autor del artículo, `p_title` de tipo `articles.title%TYPE`, también de entrada, que corresponde al título del artículo, `p_text` de tipo `articles.article_text%TYPE`, de entrada, que es el contenido del artículo, y `p_id` de tipo `articles.article_id%TYPE`, que es de salida y devolverá el id generado para el artículo.

Dentro del bloque primero se calcula el nuevo id con la sentencia `SELECT NVL(MAX(article_id), 0) + 1 INTO p_id FROM articles`. El resultado se guarda en

el parámetro de salida p_id mediante la cláusula INTO. Este cálculo funciona como alternativa a IDENTITY.

Después se ejecuta la sentencia INSERT INTO articles (article_id, user_id, title, article_date, article_text) VALUES (p_id, p_user_id, p_title, sysdate, p_text), que inserta una nueva fila en la tabla articles con el id calculado, el usuario autor, el título, la fecha y el texto recibidos.

```
CREATE OR REPLACE PROCEDURE article_update (  
    p_id    IN articles.article_id%TYPE,  
    p_title IN articles.title%TYPE,  
    p_text  IN articles.article_text%TYPE  
)  
IS  
    no_articulo EXCEPTION;  
BEGIN  
    UPDATE articles  
        SET title = p_title,  
            article_text = p_text  
        WHERE article_id = p_id;  
  
    IF SQL%ROWCOUNT = 0 THEN  
        RAISE no_articulo;  
    END IF;  
EXCEPTION  
    WHEN no_articulo THEN  
        raise_application_error(-20002, 'El artículo ' || p_id || ' no existe.');
```

Se crea el procedimiento article_update para actualizar el título y el texto de un artículo existente en la tabla articles. Recibe tres parámetros: p_id de tipo articles.article_id%TYPE, que es de entrada y es el id del artículo que se quiere actualizar, p_title de tipo articles.title%TYPE también es de entrada, que es el nuevo título, y p_text de tipo articles.article_text%TYPE de entrada. Antes del BEGIN se declara una excepción llamada no_articulo con la sentencia no_articulo EXCEPTION, que se usa por si el artículo no existe.

Dentro del bloque se ejecuta la sentencia UPDATE articles SET title = p_title, article_text = p_text WHERE article_id = p_id. Esta sentencia modifica la fila cuyo article_id coincide con el parámetro recibido, asignando el nuevo título y el nuevo texto. El WHERE es fundamental, porque sin él se actualizarían todas las filas de la tabla.

Después se verifica si la actualización afectó alguna fila usando la condición IF SQL%ROWCOUNT = 0 THEN. SQL%ROWCOUNT nos ayuda en saber

cuantas filas fueron afectadas por el comando que usamos. Si es 0, significa que no se encontró ningún artículo con ese id y entonces se lanza la excepción

```
CREATE OR REPLACE PROCEDURE article_delete (  
    p_id IN articles.article_id%TYPE  
)  
IS  
    no_articulo EXCEPTION;  
BEGIN  
  
    DELETE FROM comments WHERE article_id = p_id;  
    DELETE FROM article_tags WHERE article_id = p_id;  
    DELETE FROM article_categories WHERE article_id = p_id;  
  
    DELETE FROM articles WHERE article_id = p_id;  
    IF SQL%ROWCOUNT = 0 THEN  
        RAISE no_articulo;  
    END IF;  
EXCEPTION  
    WHEN no_articulo THEN  
        raise_application_error(-20002, 'El artículo ' || p_id || ' no existe.');
```

END;
/

Se crea el procedimiento article_delete para eliminar un artículo de la tabla articles junto con todos sus datos relacionados. Recibe un parámetro p_id de tipo articles.article_id%TYPE, que es de entrada y corresponde al id del artículo que se quiere eliminar. Antes del BEGIN se declara una excepción llamada no_articulo con la sentencia no_articulo EXCEPTION

Dentro del bloque primero se borran manualmente las filas hijas para evitar una violación de llave foránea. Se ejecutan tres sentencias DELETE: DELETE FROM comments WHERE article_id = p_id, que borra todos los comentarios del artículo. DELETE FROM article_tags WHERE article_id = p_id, que borra todas las cosas entre ese artículo y sus etiquetas. Y DELETE FROM article_categories WHERE article_id = p_id, que borra todas las relaciones entre ese artículo y sus categorías.

Después se hace DELETE FROM articles WHERE article_id = p_id. Luego se verifica si el borrado afectó alguna fila con la condición IF SQL%ROWCOUNT = 0 THEN. SQL%ROWCOUNT indica cuántas filas

fueron afectadas por la última sentencia DML. Si es 0, significa que no existía ningún artículo con ese id y entonces se lanza la excepción

```
CREATE OR REPLACE PROCEDURE article_get (  
    p_id    IN  articles.article_id%TYPE,  
    p_title OUT articles.title%TYPE,  
    p_date  OUT articles.article_date%TYPE,  
    p_text  OUT articles.article_text%TYPE,  
    p_user_id OUT articles.user_id%TYPE  
)  
IS  
BEGIN  
    SELECT title, article_date, article_text, user_id  
        INTO p_title, p_date, p_text, p_user_id  
        FROM articles  
        WHERE article_id = p_id;  
EXCEPTION  
    WHEN NO_DATA_FOUND THEN  
        raise_application_error(-20002, 'El artículo ' || p_id || ' no existe.');
```

END;
/

El article_get se usa para obtener los datos de un artículo a partir de su ide. Recibe cinco parámetros: p_id de tipo articles.article_id%TYPE, que es de entrada y corresponde al id del artículo que se quiere buscar, y cuatro parámetros de salida: p_title de tipo articles.title%TYPE, que devolverá el título del artículo, p_date de tipo articles.article_date%TYPE, que devolverá la fecha del artículo, p_text de tipo articles.article_text%TYPE, que devolverá el contenido del artículo, y p_user_id de tipo articles.user_id%TYPE, que devolverá el id del usuario autor del artículo.

Esta consulta busca en la tabla articles la fila donde article_id coincide con el id recibido y guarda los valores de las columnas en los parámetros de salida.

Si el SELECT INTO no devuelve ninguna fila y Oracle lanza automáticamente la excepción NO_DATA_FOUND. Para manejarla se usa la sección EXCEPTION con WHEN NO_DATA_FOUND THEN, que llama a raise_application_error con el código -20002 y el mensaje.

```
CREATE OR REPLACE PROCEDURE article_all (  

```

```

    p_cursor OUT SYS_REFCURSOR
)
IS
    opsql VARCHAR2(255);
BEGIN
    opsql := 'SELECT article_id, title, article_date, user_id FROM articles ORDER BY
article_id';
    OPEN p_cursor FOR opsql;
END;
/

```

Se crea el procedimiento article_all para obtener todos los artículos de la tabla articles. Recibe un único parámetro p_cursor de tipo SYS_REFCURSOR, que es de salida y devuelve un cursor con los resultados.

Antes del BEGIN se declara la variable local opsql de tipo VARCHAR2(255), que se usa para almacenar la sentencia SQL que se va a ejecutar.

Se asigna a opsql la cadena SELECT article_id, title, article_date, user_id FROM articles ORDER BY article_id. Luego se ejecuta la sentencia OPEN p_cursor FOR opsql, que abre el cursor de salida y lo asocia con el resultado de esa consulta

```

CREATE OR REPLACE PROCEDURE article_all_ordenado (
    p_atributo IN VARCHAR2,
    p_tipo    IN VARCHAR2,
    p_cursor  OUT SYS_REFCURSOR
)
IS
    v_col VARCHAR2(20);
    v_dir VARCHAR2(4);
    opsql VARCHAR2(500);
BEGIN
    IF UPPER(p_atributo) = 'TITLE' THEN
        v_col := 'title';
    ELSIF UPPER(p_atributo) = 'ARTICLE_DATE' THEN
        v_col := 'article_date';
    ELSE
        v_col := NULL;
    END IF;

    IF p_tipo = '1' THEN
        v_dir := 'ASC';
    ELSIF p_tipo = '2' THEN
        v_dir := 'DESC';
    ELSE

```

```

        v_dir := NULL;
    END IF;

    IF v_col IS NULL OR v_dir IS NULL THEN
        raise_application_error(-20003, 'Atributo o tipo de orden inválido.');
```

```

    END IF;

    opsql := 'SELECT article_id, title, article_date, user_id FROM articles ORDER BY ' || v_col
    || ' ' || v_dir;
    OPEN p_cursor FOR opsql;
END;
/
```

Se crea el procedimiento article_all_ordenado para obtener todos los artículos de la tabla articles. Recibe tres parámetros: p_atributo de tipo VARCHAR2,, p_tipo de tipo VARCHAR2 de entrada, que es el tipo de orden, y p_cursor de tipo SYS_REFCURSOR, que es de salida y devolverá el cursor con el resultado ya ordenado.

Antes del BEGIN se declaran tres variables locales: v_col de tipo VARCHAR2(20), que almacenará el nombre de la columna por la que se ordenará, v_dir de tipo VARCHAR2(4), que almacenará la dirección del orden, y opsql de tipo VARCHAR2(500), que almacenará la sentencia SQL final.

Primero se valida el atributo con una estructura IF. Si el atributo es TITLE se asigna title a v_col, si es ARTICLE_DATE se asigna article_date, y si no es ninguno de los dos se asigna NULL. Luego se valida el tipo de orden con otra estructura IF. Si p_tipo es 1 se asigna ASC a v_dir, si es 2 se asigna DESC, y cualquier otro valor se marca como NULL.

Despues con un IF se checa v_col IS NULL OR v_dir IS NULL THEN si algunas de las dos variables quedó nula, osea si el atributo o el tipo no eran válidos. En ese caso se llama a raise_application_error con el código -20003

Si todo esta bien se construye la consulta con opsql := SELECT article_id, title, article_date, user_id FROM articles ORDER BY seguido de v_col, un espacio y v_dir.

COMENTARIOS

```
CREATE OR REPLACE PROCEDURE comment_insert (
```

```

    p_article_id IN comments.article_id%TYPE,
    p_name       IN comments.name%TYPE,
    p_url        IN comments.url%TYPE,
    p_id         OUT comments.comment_id%TYPE
)
IS
BEGIN
    SELECT NVL(MAX(comment_id), 0) + 1 INTO p_id FROM comments;

    INSERT INTO comments (comment_id, article_id, name, url, comment_date)
    VALUES (p_id, p_article_id, p_name, p_url, sysdate);
END;
/

```

comment_insert se usa para insertar un nuevo comentario en la tabla **comments**. Recibe cuatro parámetros: **p_article_id** de tipo **comments.article_id%TYPE**, que es de entrada y corresponde al id del artículo al que pertenece el comentario. **p_name** de tipo **comments.name%TYPE** de entrada, es el nombre de quien comenta. **p_url** de tipo **comments.url%TYPE**, de entrada, que corresponde a un enlace del comentario. y **p_id** de tipo **comments.comment_id%TYPE**, que es de salida y devolverá el id generado

Primero se calcula el nuevo id con la sentencia **SELECT** El resultado se guarda en el parámetro de salida **p_id** con **INTO**. Después se ejecuta la sentencia **INSERT INTO comments (comment_id, article_id, name, url, comment_date) VALUES (p_id, p_article_id, p_name, p_url, sysdate)**, que inserta una nueva fila en la tabla **comments** con el identificador calculado, el artículo al que pertenece, el nombre de quien comenta, el enlace opcional y la fecha actual.

```

CREATE OR REPLACE PROCEDURE comment_all (
    p_article_id IN comments.article_id%TYPE,
    p_cursor     OUT SYS_REFCURSOR
)
IS
    opsql VARCHAR2(255);
BEGIN
    opsql := 'SELECT comment_id, name, url, comment_date FROM comments WHERE
    article_id = :aid ORDER BY comment_date';
    OPEN p_cursor FOR opsql USING p_article_id;
END;
/

```

Se crea `comment_all` para obtener todos los comentarios de un artículo específico. Recibe dos parámetros: `p_article_id` de tipo `comments.article_id%TYPE`, y `p_cursor` de tipo `SYS_REFCURSOR`, que es de salida y devolverá el cursor con el conjunto de resultados. Antes del `BEGIN` se declara la variable local `opsql` de tipo `VARCHAR2(255)`, que se usará para almacenar la sentencia SQL que se va a ejecutar.

Se asigna a `opsql` la cadena `SELECT comment_id, name, url, comment_date FROM comments WHERE article_id = :aid ORDER BY comment_date`. En esta cadena aparece la variable de enlace `:aid`, que es un marcador de posición que luego se reemplaza por el valor real del parámetro.

Después abre el cursor de salida y lo asocia con la consulta almacenada en `opsql`. La cláusula `USING` le da un valor a `:aid` que en este caso es el parámetro `p_article_id`.

```
CREATE OR REPLACE FUNCTION fn_contar_comentarios (  
    p_article_id IN comments.article_id%TYPE  
) RETURN NUMBER  
IS  
    v_count NUMBER;  
BEGIN  
    SELECT COUNT(*) INTO v_count  
    FROM comments  
    WHERE article_id = p_article_id;  
    RETURN v_count;  
END;  
/
```

Se usa `fn_contar_comentarios` para contar cuántos comentarios tiene un artículo específico en la tabla `comments`. Recibe un parámetro `p_article_id` de tipo `comments.article_id%TYPE`, que es de entrada y corresponde al `id` del artículo con los comentarios que se quieren contar. Se declara `RETURN NUMBER`, lo que dice que devolverá un valor numérico. Antes del `BEGIN` se declara la variable local `v_count` de tipo `NUMBER`, que se usará para guardar el resultado del conteo.

Se ejecuta la sentencia `SELECT COUNT(*) INTO v_count FROM comments WHERE article_id = p_article_id`. La función `COUNT` cuenta cuántas filas de la tabla `comments` cumplen la condición. El resultado del conteo se guarda en la variable `v_count`.

Luego ejecuta **RETURN v_count**, que devuelve el valor al llamador.

TAGS

```
CREATE OR REPLACE PROCEDURE tag_insert (  
    p_name IN tags.name%TYPE,  
    p_url  IN tags.url%TYPE,  
    p_id   OUT tags.tag_id%TYPE  
)  
IS  
BEGIN  
    SELECT NVL(MAX(tag_id), 0) + 1 INTO p_id FROM tags;  
    INSERT INTO tags (tag_id, name, url) VALUES (p_id, p_name, p_url);  
END;  
/
```

tag_insert se usa para insertar una nueva etiqueta en la tabla tags.

Recibe tres parámetros: **p_name** de tipo [tags.name%TYPE](#). **p_url** de tipo [tags.url%TYPE](#). y **p_id** de tipo [tags.tag_id%TYPE](#), que es el id asociado

Primero se calcula el nuevo id con la sentencia **SELECT NVL(MAX(tag_id), 0) + 1 INTO p_id FROM tags**. El resultado se guarda en el parámetro de salida **p_id** con la cláusula **INTO**. Este cálculo funciona como alternativa a **IDENTITY**.

Se ejecuta **INSERT INTO tags (tag_id, name, url) VALUES (p_id, p_name, p_url)**, que inserta una nueva fila en la tabla tags con el id calculado, el nombre y el enlace recibidos.

```
CREATE OR REPLACE PROCEDURE tag_associate (  
    p_article_id IN article_tags.article_id%TYPE,  
    p_tag_id     IN article_tags.tag_id%TYPE  
)  
IS  
    ellave_primaria EXCEPTION;  
    PRAGMA exception_init(ellave_primaria, -00001);  
BEGIN  
    INSERT INTO article_tags (article_id, tag_id) VALUES (p_article_id, p_tag_id);  
EXCEPTION  
    WHEN ellave_primaria THEN  
        raise_application_error(-20004, 'Ese tag ya está asociado al artículo.');
```

tag_associate para darle una etiqueta a un artículo insertando una nueva fila en la tabla **article_tags**. Recibe dos parámetros: **p_article_id** de tipo **article_tags.article_id%TYPE**, **p_tag_id** de tipo **article_tags.tag_id%TYPE**, también de entrada, que corresponde al id de la etiqueta.

Antes del **BEGIN** se declara una excepción que es una violación de llave primaria

Luego ponemos una sentencia que inserta la relación entre el artículo y la etiqueta.

En la parte de **EXCEPTION** se captura esa excepción con **WHEN** **ellave_primaria** **THEN** y se llama a **raise_application_error**

```
CREATE OR REPLACE PROCEDURE tags_by_article (  
    p_article_id IN tags.tag_id%TYPE,  
    p_cursor OUT SYS_REFCURSOR  
)  
IS  
    opsql VARCHAR2(500);  
BEGIN  
    opsql := 'SELECT t.tag_id, t.name, t.url FROM tags t JOIN article_tags atg ON atg.tag_id  
= t.tag_id WHERE atg.article_id = :aid ORDER BY t.name';  
    OPEN p_cursor FOR opsql USING p_article_id;  
END;  
/
```

Se crea el procedimiento **tags_by_article** para obtener todas las etiquetas asociadas a un artículo específico. Recibe dos parámetros: **p_article_id**, y corresponde al identificador del artículo cuyas etiquetas se quieren obtener, y **p_cursor** de tipo **SYS_REFCURSOR** y devuelve el cursor con los resultados

Antes del **BEGIN** se declara la variable local **opsql** de tipo **VARCHAR2(500)**,

opsql toma la cadena **SELECT t.tag_id, t.name, t.url FROM tags t JOIN article_tags atg ON atg.tag_id = t.tag_id WHERE atg.article_id = :aid ORDER BY t.name**. **WHERE atg.article_id = :aid** limita el resultado al artículo que solicitamos, y **ORDER BY t.name** ordena las etiquetas por nombre.

Después se ejecuta la sentencia OPEN p_cursor FOR opsq1 USING p_article_id, que abre el cursor de salida y lo junta con la consulta almacenada en opsq1. USING le da el valor a :aid, que en este caso es p_article_id. A partir de ese momento el llamador puede recorrer todas las filas del resultado.

```
CREATE OR REPLACE PROCEDURE articles_by_tag (  
    p_tag_name IN tags.name%TYPE,  
    p_cursor OUT SYS_REFCURSOR  
)  
IS  
    opsq1 VARCHAR2(500);  
BEGIN  
  
    opsq1 := 'SELECT a.article_id, a.title, a.article_date FROM articles a ' ||  
        'JOIN article_tags atg ON atg.article_id = a.article_id ' ||  
        'JOIN tags t ON t.tag_id = atg.tag_id ' ||  
        'WHERE LOWER(t.name) = LOWER(:tname) ORDER BY a.article_date DESC';  
    OPEN p_cursor FOR opsq1 USING p_tag_name;  
END;  
/
```

se usa articles_by_tag para obtener todos los artículos que tienen indicadas una etiqueta. Recibe dos parámetros: p_tag_name de tipo tags.name%TYPE, y p_cursor de tipo SYS_REFCURSOR. Antes del BEGIN se declara la variable local opsq1 de tipo VARCHAR2(500), se asigna a opsq1 una cadena. Después se ejecuta la sentencia OPEN p_cursor FOR opsq1 USING p_tag_name, que abre el cursor que se junta con la consulta almacenada en opsq1. La cláusula USING proporciona el valor para la variable de enlace :tname, que en este caso es el parámetro p_tag_name.

CATEGORIAS

```
CREATE OR REPLACE PROCEDURE category_insert (  
    p_name IN categories.name%TYPE,  
    p_url IN categories.url%TYPE,  
    p_id OUT categories.category_id%TYPE  
)  
IS
```

```

BEGIN
    SELECT NVL(MAX(category_id), 0) + 1 INTO p_id FROM categories;
    INSERT INTO categories (category_id, name, url) VALUES (p_id, p_name, p_url);
END;
/

```

category_insert se usa para insertar una nueva categoría en la tabla **categories**. Recibe tres parámetros: **p_name** de tipo **categories.name%TYPE**, y **p_id** de tipo **categories.category_id%TYPE**
Se calcula en nuevo id

El resultado se guarda en el parámetro de salida p_id con INTO
Después se ejecuta la sentencia INSERT INTO que inserta una nueva fila en la tabla categories con el id, calculado, el nombre y el enlace recibidos.

```

CREATE OR REPLACE PROCEDURE category_associate (
    p_article_id IN article_categories.article_id%TYPE,
    p_category_id IN article_categories.category_id%TYPE
)
IS
    ellave_primaria EXCEPTION;
    PRAGMA exception_init(ellave_primaria, -00001);
BEGIN
    INSERT INTO article_categories (article_id, category_id)
    VALUES (p_article_id, p_category_id);
EXCEPTION
    WHEN ellave_primaria THEN
        raise_application_error(-20005, 'Esa categoría ya está asociada al artículo.');
```

category_associate se usa para asociar una categoría a un artículo insertando una nueva fila en la tabla **article_categories**. Recibe dos parámetros: **p_article_id** de tipo **article_categories.article_id%TYPE** que es **p_category_id**.

Antes del BEGIN se declara una excepción llamada ellave_primaria y se asocia al código de error -00001 con PRAGMA exception_init(ellave_primaria, -00001).

Se ejecuta INSERT INTO article_categories (article_id, category_id) VALUES (p_article_id, p_category_id)

En EXCEPTION se captura esa excepción con WHEN ellave_primaria THEN y se llama a raise_application_error con el código -20005 y el mensaje Esa categoría ya está asociada al artículo.

```

CREATE OR REPLACE PROCEDURE categories_by_article (

```

```

    p_article_id IN article_categories.article_id%TYPE,
    p_cursor OUT SYS_REFCURSOR
)
IS
    opsql VARCHAR2(500);
BEGIN
    opsql := 'SELECT c.category_id, c.name, c.url FROM categories c ' ||
        'JOIN article_categories ac ON ac.category_id = c.category_id ' ||
        'WHERE ac.article_id = :aid ORDER BY c.name';
    OPEN p_cursor FOR opsql USING p_article_id;
END;
/

```

Se crea el procedimiento categories_by_article para obtener todas las categorías asociadas a un artículo específico. Recibe dos parámetros:

p_article_id de tipo article_categories.article_id%TYPE y p_cursor de tipo SYS_REFCURSOR, que devolverá el cursor con los resultados

Antes del BEGIN se declara la variable local opsql de tipo VARCHAR2(500),

se asigna a opsql una cadena SQL

Después se ejecuta la sentencia OPEN p_cursor FOR opsql USING p_article_id, que abre el cursor de salida y lo asocia con la consulta almacenada en opsql. USING le da el valor para la variable de enlace :aid, que en este caso es p_article_id. Aquí el llamador puede recorrer todas las filas del resultado, que son las categorías del artículo indicado ordenadas por nombre.

Repositorio en GitHub:

[a376899-c/Blog-Oracle-PLSQL: Aplicación de administración de un blog desarrollada en Python, Oracle y PL/SQL.](#)